

TRABALHO PRÁTICO 2023

2º SEMESTRE

Complementos de Programação de Computadores
Docentes: Ricardo Manuel Carrola Simões & Tiago Pereira Rodrigues



Francisca Ferreira Domingues

Número: A 104953

a104953@alunos.uminho.pt



Isabel Carvalho Gomes

Número: A104868

a104868@alunos.uminho.pt



Joana Martins de Carvalho

Número: A104865

a104865@alunos.uminho.pt

Resumo

Neste trabalho serão apresentados um código e um relatório, como base de suporte ao jogo por nós elaborado.

O relatório inclui uma explicação sobre as características do jogo retro, em causa, o R-Type. O qual apresenta enumeras semelhanças ao nosso jogo final. Serão ainda explicados os diversos blocos de código à medida que são implementados.

Quanto ao código criado, permite a visualização de toda a estrutura do jogo tal como as suas normas, as quais serão abordadas posteriormente.



Índice

Capa.....	1
Resumo.....	3
Introdução.....	5
Descrição do Problema.....	6
Arquitetura do Sistema.....	7
Implementação do Jogo.....	8
▪ Representação do Estado do Jogo;	
▪ Inicialização do Estado do Jogo;	
▪ Visualização do Estado do Estado do Jogo;	
▪ Mudança de Estado/ Movimento do Jogador;	
▪ Salvar e Restaurar Jogo;	
▪ Final da Execução;	
▪ Software utilizado.	
Conclusões e Perspetivas de Desenvolvimento.....	21
Bibliografia.....	22

Introdução

No jogo R-Type, à semelhança do apresentado por nós o jogador controla os movimentos de uma nave espacial artilhada com um canhão, que se movimenta na parte esquerda do ecrã. Na parte direita, estão posicionados aliens, que se deslocam até às extremidades (esquerda e direita) do ecrã. O objetivo do jogador é evitar que os aliens o alcancem na sua secção de atuação. Para essa tarefa o canhão possui munição infinita para disparar.

Ao destruir um número grande aliens, os restantes começam a marchar mais rapidamente em direção à nave humana. Quando todos os aliens são eliminados, um novo pelotão é constituído, iniciando uma linha abaixo da formação anterior.

Em termos estruturais, espera-se que o jogo integre, na sua base, uma interface gráfica retro simplista. O desenvolvimento da aplicação explora alguns dos conceitos dados nas aulas de CPC.

Descrição do Problema

Desde a manipulação de variáveis auxiliares simples, ciclos e estruturas de decisão, funções para organização e reutilização de código, matrizes – por exemplo, para a configuração dos pelotões de aliens – estruturas de dados e classes para guardar informação relevante, escrita e leitura de ficheiros para o alojamento em memória permanente do estado do jogo e do carregamento do mesmo, respetivamente.

Explicações dos elementos do código:

#pragma once: É uma diretiva de pré-processador que garante que o cabeçalho seja incluído apenas uma vez no programa.

#include: Diretivas para incluir os cabeçalhos necessários para o código, como Allegro e algumas definições.

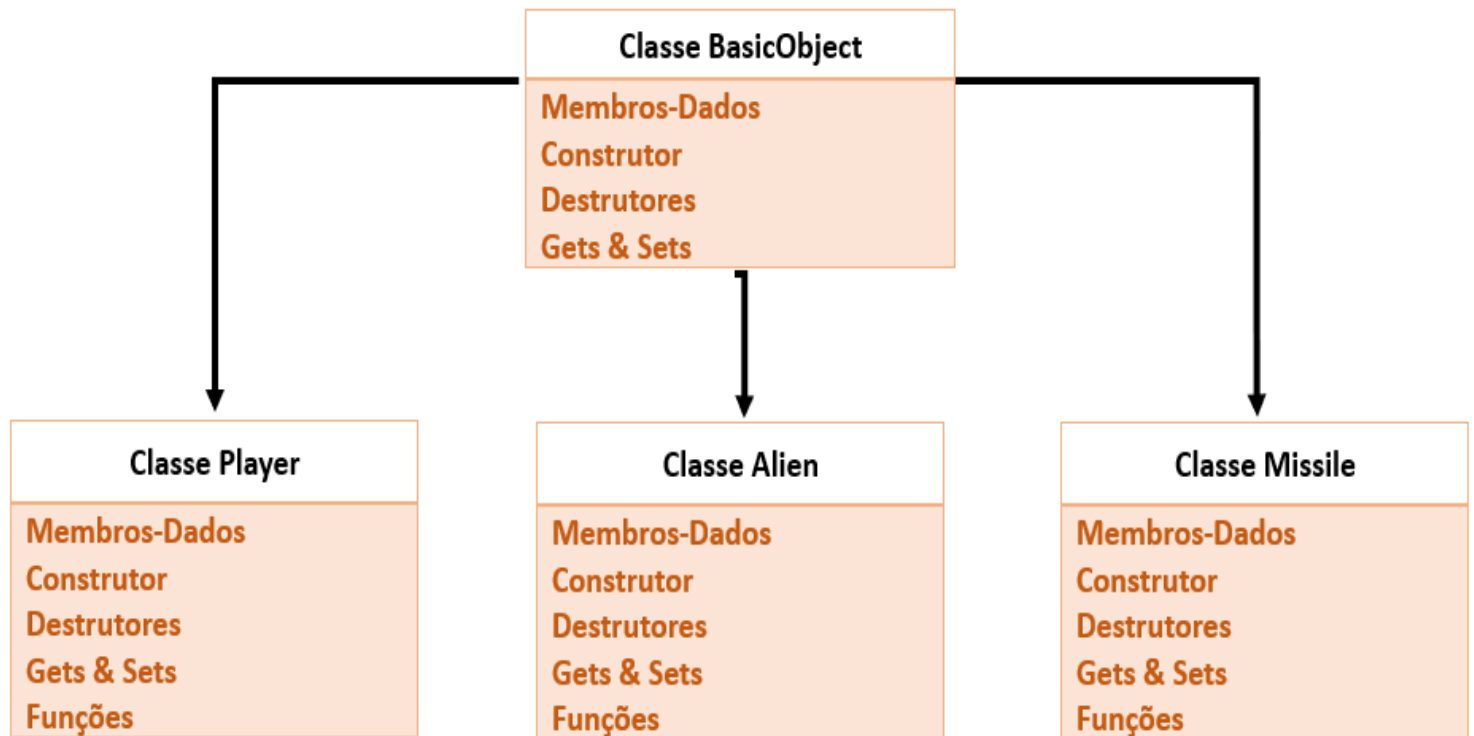
Declaração de membros de uma classe:

protected: a própria classe tem acesso aos membros. Esses atributos também são acessíveis pelas classes derivadas, mas não são diretamente acessíveis por outras classes.

Private: Somente a própria classe tem acesso aos membros declarados como private. Eles são ocultos e não podem ser acedidos diretamente por outras classes ou funções externas.

Public: Os membros declarados como public são visíveis e acessíveis a partir de qualquer ponto do programa. Eles definem a interface pública da classe e podem ser acedidos e utilizados por outras classes ou funções externas.

Arquitetura do Sistema



Implementação do Jogo

CLASSES:

Para a implementação do jogo foi necessário identificar as entidades necessárias.

Entre as quais:

- Player (nave);
- Alien (bigBoss ou não);
- Missile.

Para representar um player, alien ou missile é necessário definir a sua posição (x,y), o seu tamanho, número de vidas e uma imagem definida. Uma vez que todas estas entidades necessitam dos mesmos parâmetros, foi definida uma classe básica:

- **Classe BasicObject:** contém toda as propriedades básicas para definir coordenadas, dimensões do objeto, velocidade, número de vidas e uma imagem associada.

As entidades player, alien e missile herdam da classe BasicObject. Além dos parâmetros básicos, que têm acesso da herança, cada classe tem os seus membros específicos.

- **Classe Player:** entidade que define o jogador. Contém toda a definição “básica”, um identificador (Id), uma pontuação (score) e uma coleção de misseis (vector).
- **Classe Alien:** entidade que define o alien. Contém toda a definição “básica” e um membro que define se é ou não um “bigBoss” (booleano).
- **Classe Missile:** entidade que define o Míssil disparado pelo jogador (player). Contém toda a definição “básica” e um membro que define se está ativo ou não (booleano);

NOTA: Num ficheiro extra foram definidas as dimensões da tela, as dimensões da área de jogo y o frameTime (FPS – frame per second).

ESTRATÉGIAS DE JOGO:

O alien elimina-se em 3 situações:

- quando ultrapassa a tela pelo lado esquerdo;
- quando colide com um míssil;
- quando colide com o jogador.

REGRAS:

Gestão de vidas:

- O jogador inicia o jogo com 5 vidas.
- O alien normal é definido com 1 vida e o alien bigBoss com 3.
- O jogador perde 1 vida quando colide com um alien normal;
- O jogador perde o jogo quando colide com o alien bigBoss.

Gestão de pontos:

- Destruição de um alien normal: +1 ponto;
- Míssil colide no alien bigBoss: +2 pontos;
- Destruição de um alien bigBoss: +6 pontos;
- Alien atravessa lado esquerdo da tela: -2 pontos.

❖ Representação do Estado do Jogo

Classe BasicObject

Header File

Atributos:

- int x, y, width, height, lives: representam as coordenadas, largura, altura e vidas do objeto;
- float speed: representa a velocidade do objeto;
- ALLEGRO_BITMAP* image: Declaração do atributo imagem, que é um ponteiro para ALLEGRO_BITMAP e armazena a imagem do objeto.

CPP

- #include "../Includes/basicObject.h": é incluído para fornecer as definições da classe BasicObject.
- nullptr é usada para inicializar o ponteiro imagem com um valor nulo. Isso significa que, inicialmente, o objeto BasicObject não possui uma imagem associada a ele.

Métodos:

- BasicObject (): O construtor padrão da classe BasicObject;
- BasicObject(int f_x,int f_y,int f_lives,float f_speed,ALLEGRO_BITMAP* f_image): Um construtor que permite definir as propriedades básicas (posição, vidas, velocidade e uma imagem associada);
- ~ BasicObject (): O destrutor da classe BasicObject. Responsável por liberar recursos associados à classe;
- Métodos setter/getter: para definir/obter os membros da classe.

Classe Alien

Header File

A classe Alien é derivada da classe BasicObject.

Atributos:

- bool bBigBoss: Uma variável booleana que indica se o alienígena é um chefe ("big boss") ou não;
- int scale = 7: Uma variável inteira que representa a escala do alienígena normal, com o valor padrão de 7;
- int scaleBoss = 3: Uma variável inteira que representa a escala do alienígena biBoss, com o valor padrão de 3;
- bool goingUp: Uma variável booleana que indica a direção (cima/baixo) do primeiro movimento.

Métodos:

- Alien(): O construtor padrão da classe Alien;
- Alien(bool f_isBigBoss, int f_x, int f_y, int f_lives, float f_speed, bool f_goingUp, ALLEGRO_BITMAP* f_image): Um construtor que permite definir as propriedades iniciais do alienígena. Se é um "big boss", posição, vidas, velocidade, sentido do primeiro movimento e uma imagem associada;
- ~Alien(): O destrutor da classe Alien.
- void setIsBigBoss(bool f_isBigBoss): Um método para definir se o alienígena é um "big boss" ou não;
- bool isBigBoss(): Um método para obter se o alienígena é um "big boss" ou não.
- void draw(): Um método para desenhar o alienígena;
- void updatePosition(): Um método para atualizar a posição do alienígena.

CPP

- `#include "../Includes/alien.h"` é incluído para fornecer as definições da classe `Alien`;
- `Alien::Alien()`: Construtor padrão da classe `Alien`. Inicializa a classe base `BasicObject` e define o atributo `bBigBoss` como `false`, indicando que o alienígena não é um chefe;
- `Alien::~~Alien()`: Destrutor da classe `Alien`;
- `Alien::Alien(bool f_isBigBoss, int f_x, int f_y, int f_lives, float f_speed, bool f_goingUp, ALLEGRO_BITMAP* f_image)`: Construtor que permite definir as propriedades iniciais do alienígena. Ele inicializa a classe base `BasicObject` com as informações de posição, vidas, velocidade e imagem. Além disso, define o atributo `bBigBoss` com o valor fornecido e calcula a largura e altura do alienígena com base nas dimensões da imagem. Define se o primeiro movimento do alien é para cima ou para baixo com o valor de `f_goingUp`;
- `Alien::setIsBigBoss(bool f_isBigBoss)`: Setter que permite definir se o alienígena é um chefe ou não;
- `Alien::isBigBoss()`: Getter que retorna se o alienígena é um chefe ou não;
- `Alien::updatePosition()`: Método responsável por atualizar a posição do alienígena. Nesse caso, o alienígena move-se para a esquerda, subtraindo a velocidade atual da coordenada `x` e a variável `y` pode ser incrementada/decrementada em função da direção do movimento (cima/baixo);
- `Alien::draw()`: Método para desenhar o alienígena. Utiliza a função `al_draw_scaled_bitmap` para desenhar a imagem do alienígena.

Classe Player

Header File

A classe Alien é derivada da classe BasicObject.

- As linhas `#include` são para incluir as definições de "basicObject.h" e "missile.h", que contêm as definições das classes BasicObject e Missile, respetivamente;
- Definido um enumerador chamado `TypeOfMove`, que define os tipos de movimento possíveis para o jogador. Os valores possíveis são UP (cima), DOWN (baixo), LEFT (esquerda), RIGHT (direita) e STOP (parar);

Atributos: id (ID do jogador), score (pontuação do jogador) e `vctrMissiles` (vetor de mísseis disparados pelo jogador).

Métodos:

- Os **construtores** `Player()` e `Player(...)` são responsáveis por inicializar os atributos da classe;
- O **destrutor** `~Player()`;
- Os métodos `draw()`, `updatePosition()`, `drawLives()` e `fire()` são responsáveis por desenhar o jogador, atualizar a sua posição, desenhar as suas vidas e disparar um míssil, respetivamente;
- Os métodos **setters** `setId()` e `setScore()` são utilizados para definir o ID e a pontuação do jogador, respetivamente;
- Os métodos **getters** `getId()` e `getScore()` são utilizados para obter o ID e a pontuação do jogador, respetivamente.

- `#include "../Includes/player.h"` é incluído para fornecer as definições da classe `Player`;
- Construtor padrão `Player::Player()` inicializa o objeto `Player` chamando o construtor da classe `BasicObject`. Define os valores iniciais para `id` e `score`, e limpa o vetor `vctrMissilesL`;
- Destrutor `Player::~~Player()`;
- Construtor `Player::Player(...)` inicializa o objeto `Player` chamando o construtor da classe `BasicObject`. Define os valores iniciais para `id` e `score` com base nos argumentos fornecidos. Limpa o vetor `vctrMissiles`. Calcula a escala do objeto `Player` em relação à imagem fornecida;
- Metodo `Player::setId(int f_id)`: Define o ID do jogador;
- Metodo `Player::setScore(int f_score)`: Define a pontuação do jogador;
- Metodo `Player::getId()`: Retorna o ID do jogador;
- Metodo `Player::getScore()`: Retorna a pontuação do jogador;
- Metodo `Player::draw()`: Desenha o jogador na tela usando a função `al_draw_scaled_bitmap` para dimensionar a imagem do jogador;
- Metodo `Player::updatePosition(OfTypeMove f_move)`: Atualiza a posição do jogador com base no tipo de movimento fornecido;
- Metodo `Player::drawLives()`: Desenha as vidas do jogador na tela;
- Metodo `Player::fire(ALLEGRO_BITMAP* f_image)`: Verifica se o vetor `vctrMissiles` está vazio. Se não estiver vazio, verifica se a posição do último míssil no vetor está distante o suficiente do jogador para disparar um novo míssil. Se sim, cria um objeto `Missile` e adiciona-o ao vetor `vctrMissiles`. Se o vetor `vctrMissiles` estiver vazio, cria um objeto `Missile` e adiciona-o ao vetor `vctrMissiles`.

Classe Missile

Header File

A classe Alien é derivada da classe BasicObject.

Atributos:

- bool enable indica se o míssil está habilitado ou não. Quando o míssil está habilitado, ele pode se mover e ser desenhado na tela;
- int scale = 14 define a escala do míssil em relação à imagem.

Métodos:

- Missile(); É o construtor padrão da classe Missile.
- Missile(bool f_enable, int f_x, int f_y, int f_lives, float f_speed, ALLEGRO_BITMAP* f_image) construtor que recebe vários parâmetros para inicializar um míssil com valores específicos. Os parâmetros incluem o estado de habilitação do míssil, a posição inicial, o número de vidas, a velocidade e a imagem do míssil;
- ~Missile() destrutor da classe Missile;
- void draw() função que desenha o míssil na tela;
- void updatePosition() função que atualiza a posição do míssil;
- void setEnable(bool f_enable) função que define se o míssil está habilitado ou não;
- int isEnabled(): É uma função que verifica se o míssil está habilitado. Ela retorna um valor inteiro indicando se o míssil está habilitado (1) ou não (0).

- `#include "../Includes/missile.h"` é incluído para fornecer as definições da classe `Missile`;
- O construtor padrão `Missile::Missile()` é definido. Ele inicializa a classe `Missile` e chama o construtor da classe `BasicObject` (padrao). A variável `enable` é inicializada com o valor `false`. Essa variável controla o estado de ativação do míssil;
- O construtor `Missile::Missile()` é definido com parâmetros. Ele permite definir valores iniciais para todas as propriedades do míssil. O construtor chama o construtor da classe `BasicObject` (com parâmetros) com os parâmetros relevantes para inicializar as propriedades do objeto base. A variável `enable` é definida com o valor passado no parâmetro `f_enable`;
- `~Missile()` destrutor da classe `Missile`;
- Método `Missile::setEnabled()`: permite definir o estado de ativação do míssil através do parâmetro `f_enable`;
- Método `Missile::isEnabled()`: retorna o estado de ativação atual do míssil;
- Método `Missile::draw()`: desenha o míssil na tela devidamente dimensionado;
- Método `Missile::updatePosition()`: atualiza a posição do míssil com base em sua velocidade. Neste caso, a posição `x` é incrementada pela velocidade do míssil, fazendo com que ele se mova para a direita.



Inicialização do Estado do Jogo

Todas as variáveis de jogo são inicializadas desde o código: as definições dos elementos do jogo (jogador, aliens e bigBoss), o espaço navegável e as pontuações.

Apenas se define um jogador no lado esquerdo da tela e centrado no eixo Y do espaço jogável.

Os aliens são gerados em função da pontuação do jogador. Todos os parâmetros envolvidos no processo de gerar aliens é definido no código.

Na sua criação, define-se a sua posição inicial e o sentido de movimento na vertical (o sentido em X é sempre constante, da direita para esquerda). A sua posição inicial em Y, bem como o sentido de movimento em Y, são gerados aleatoriamente. Esse movimento é limitado pela altura do espaço navegável, tendo sempre uma trajetória diagonal. A cada alien gerado é adicionado a um vector que se modifica, durante a execução do jogo.

- $0 \leq \text{pontuação} < 25$ pontos: aliens com velocidade padrão;
- $25 \leq \text{pontuação} < 50$ pontos: são gerados mais aliens e apresentam maior velocidade;
- 50 pontos: é gerado o alien bigBoss, apenas quando os restantes aliens forem eliminados. O objetivo é ter uma batalha 1 vs 1.

Propriedades do jogo (dimensões da tela e espaço jogável) são definidos num ficheiro externo (definitions.h). A classe BasicObject inclui esse ficheiro.

❖ **Visualização do Estado do Estado do Jogo**

Durante o jogo foi adicionado um background para representar o espaço.

O jogador é identificado por uma nave (com orientação Este), os aliens normais por naves pequenas (orientação Oeste) e um bigBoss por uma nave grande (orientação Oeste). Os mísseis que são projetados pelo jogador são facilmente identificáveis. Todos os elementos do jogo estão representados por imagens.

Ao lado da imagem do alien “bigBoss” existem 3 retângulos, que representam o número de vidas restantes.

O espaço navegável está definido por um retângulo branco. Qualquer elemento do jogo, apenas se move dentro desse espaço.

Fora desse espaço é possível observar as vidas restantes do jogador, bem como a pontuação atual.

Foi adicionado uma pequena imagem a representar uma explosão quando uma colisão ocorre (player-alien, missil-alien).

Adiciona uma fonte de texto específica (PixelGameFont.ttf) para que toda a informação que é apresentada na tela tenha o efeito de jogo retro.

Foram adicionados sons para representar situações do jogo.

❖ Mudança de Estado/ Movimento do Jogador

O jogador deve manipular a nave com recurso às setas do teclado e do espaço. Cada uma das setas representa o movimento correspondente e o espaço para efetuar disparos. Em função da seta premida é determinada a nova posição (x,y), mas apenas é mostrado na tela quando se executa o método draw().

O jogador pode navegar livremente por todo o espaço navegável. A origem do referencial encontra-se no canto superior esquerdo e sua expansão dá-se no sentido do canto inferior direito. A largura e altura da janela é definida pelos parâmetros width e height, respetivamente.

Assim, para realizar o efeito de movimento é necessário modificar a posição (x,y) do elemento.

- Movimento para a direita: $x = x + \text{speed};$
- Movimento para a esquerda: $x = x - \text{speed};$
- Movimento para cima: $y = y - \text{speed};$
- Movimento para baixo: $y = y + \text{speed}.$

Cada disparo do jogador corresponde a criação de um objeto tipo Missile e, é adicionado ao vetor interno de mísseis. A este elemento apenas se atualiza a posição em x, descrevendo o movimento horizontal.

Implementada uma função que verifica colisões entre jogador/alien e míssil/alien, cada jogador/alien/míssil é definido por uma caixa e aplicam-se as condições para verificar se os limites das caixas sobrepõem em ambos os eixos X e Y.

Para atualizar o número de vidas, pontuação, mísseis, entre outros, faz-se o recurso aos métodos get/set das classes definidas.

❖ Salvar e Restaurar Jogo

Foi implementado um sistema para guardar e carregar o top score. Foram usadas as funções de manipulação de ficheiros de C++, como `std::ofstream` para escrever e `std::ifstream` para ler.

A função `saveScoreToFile()` cria um objeto `std::ofstream` chamado `file` e escreve o valor do score no ficheiro `"topScore.txt"`. A função `readScoreFromFile()` cria um objeto `std::ifstream` chamado `file` e lê o valor do score do ficheiro.

❖ Final da Execução

O jogador vence ao eliminar o alien `bigBoss`. Este alien tem 3 vidas e o jogador necessita de disparar 3 mísseis para o eliminar.

O jogador perde quando ocorre esgota as vidas ou colide com o alien `bigBoss`.

❖ Software utilizado

Biblioteca gráfica (Allegro 5).

Visual Studio 2022.

Conclusões e Perspetivas de Desenvolvimento

- A complexidade do jogo poderia expandir-se, mas a implementação atual tem a lógica necessária para um jogo funcional.
- Melhorias:
 - complexidade das rotas para os aliens;
 - adicionar disparo por parte dos aliens.
 - adicionar modo multijogador.

BIBLOGRAFIA

Allegro 5 - <https://liballeg.org/a5docs/trunk/>
<https://www.allegro.cc/manual/5/index.html>

Livros Utilizados:

Loureiro, Henrique. C++ Guia Moderno de Programação 1ª Edição. Editora de Informática, setembro 2019.

Damas, Luís. Linguagem C 24ª Edição. Editora de Informática, outubro 2015
reimpressão de janeiro 2019.

Efeitos de som - <https://mixkit.co/free-sound-effects/>

Link da imagem da página 3 - https://fs-prod-cdn.nintendo-europe.com/media/images/10_share_images/games_15/virtual_console_wii_u_7/H2x1_WiiUVC_RType_image1600w.jpg